Mavi VPN Technical Whitepaper

Advanced Architecture, Protocol Engineering & Deep Packet Inspection Resistance

Mavi Development Team

April 19, 2026

Abstract

This whitepaper provides an exhaustive technical analysis of Mavi VPN, a next-generation, high-performance, censorship-resistant Virtual Private Network architecture built entirely in Rust. It details the utilization of the IETF QUIC protocol for zero-head-of-line-blocking transport, the implementation of Encrypted Client Hello (ECH) and MASQUE (RFC 9484) to defeat Deep Packet Inspection (DPI) systems, and the novel "Pinned MTU" strategy. By leveraging highly optimized asynchronous I/O frameworks, dual-stack routing dynamics, and rigorous cryptographic enforcement, Mavi VPN ensures continuous, robust connectivity across the most restrictive and constrained network environments.

Contents

1 Introduction and Core Philosophy **2**
1.1 Key Design Goals 2

2 System Architecture and Topology **2**
2.1 Architectural Component Layout 2
2.2 The Backend (Server-Side) 3
2.3 Platform-Specific Clients 3

3 Transport & Framing: The QUIC Paradigm **3**
3.1 Datagrams vs. Streams 3
3.2 Congestion Control: BBR (Bottleneck Bandwidth and RTT) 4

4 Censorship Resistance & DPI Evasion **4**
4.1 Encrypted Client Hello (ECH - RFC 9180) 4
4.2 Active Probe Emulation (The Fake Nginx Server) 4
4.3 MASQUE (RFC 9484) Connect-IP Encapsulation 5

5 Authentication and Cryptography **5**
5.1 SHA-256 Certificate Pinning 5
5.2 Identity Provider Integration (Keycloak OIDC) 5

6 The "Pinned MTU" Engineering Strategy **6**
6.1 Evaluating Path MTU Discovery (PMTUD) 6
6.2 Dual-Boundary Pinned Limitation 6
6.3 ICMP Asynchronous Emulation 6

7 System Optimization and Kernel Offloading **7**
7.1 Buffer Inflation and GSO 7
7.2 IpGuard RAII and Dual-Stack Operations 7

8 Conclusion **7**

1 Introduction and Core Philosophy

Historically, Virtual Private Networks (VPNs) such as OpenVPN, WireGuard, and IPsec were engineered with the primary assumption of operating within open, neutral networks. Their wire formats are highly recognizable; WireGuard, for example, maintains a static 148-byte handshake initiation structure that state-sponsored firewalls flag and drop instantly.

Mavi VPN discards these legacy assumptions. It operates on the principle that *all* traffic is subject to hostile interception, active probing, and Deep Packet Inspection (DPI). To combat this, Mavi VPN's data and control planes are designed to be statistically and behaviorally indistinguishable from standard HTTP/3 web traffic.

1.1 Key Design Goals

- **Imperceptible Fingerprinting**: Obfuscating VPN traffic behind legitimate HTTP/3 application-layer framing (ALPN `h3`) and SNI spoofing using Encrypted Client Hello (ECH / RFC 9180).
- **Extreme Performance**: Utilizing Rust's `tokio` asynchronous runtime, zero-copy packet slicing (`bytes` crate), and hardware-accelerated cryptography (`aws-lc-rs`) to saturate high-bandwidth links while minimizing CPU scheduling overhead.
- **Network Resilience**: Implementing dynamic connection migration to support mobile roaming (e.g., transitioning between LTE and Wi-Fi networks without dropping packets) and utilizing Bottleneck Bandwidth and Round-trip time (BBR) congestion control to optimize link utilization over high-latency connections.
- **Memory Safety without Compromise**: Eliminating remote code execution (RCE) vulnerabilities—such as buffer overflows and use-after-free conditions prevalent in C/C++ implementations—through Rust's stringent ownership and borrowing paradigms.

2 System Architecture and Topology

The ecosystem is meticulously separated into a high-performance backend, platform-specific privileged network services, and unprivileged user interfaces.

2.1 Architectural Component Layout

The architecture separates unprivileged UI tasks from highly privileged network stack manipulations using local Inter-Process Communication (IPC).

```
[ Graphical User Interface (Tauri) ] --------------+
  (Cross-Platform: Windows, Linux)                 |
                                                   | (TCP 14433 IPC)
                                                   |
+-----------------------------------+              |
|   Platform Core (Windows/Linux)   | <------------+     [ Mobile Client ]
|   Privileged: WinTUN/Linux TUN    |                    (Android Kotlin UI)
|   Route & DNS Policy Management   |                             |
+-----------------------------------+                             | (JNI)
                 |                                                |
                 +----------  QUIC Datagram Tunnel  --------------+
                 (TLS 1.3 / ChaCha20 / MASQUE Connect-IP Capsules)
                                                   |
```

```
+-----------------------+
|      VPN Server       |
|  (Rust + tokio + tun) |
| Authentication & Pkt  |
|     Routing Hub       |
+-----------------------+
```

2.2 The Backend (Server-Side)

The Linux-based backend is a horizontally scalable routing hub. It is responsible for terminatingQUIC connections, verifying identity tokens or OpenID Connect (OIDC) JWTs, allocating private Dual-Stack IP addresses, and routing packets between the inner virtual network and the internet via Network Address Translation (NAT).

To maximize throughput, the server relies on **mimalloc** as the global memory allocator. Since a VPN server handles millions of tiny ‘bytes::BytesMut‘ allocations per second per client, standard malloc implementations often suffer from extreme heap fragmentation and thread-contention. `mimalloc` guarantees constant-time allocation even under heavily multiplexed asynchronous workloads.

2.3 Platform-Specific Clients

Mavi VPN provides native drivers rather than relying on generic user-space wrappers:

- **Windows (`windows` crate)**: Uses the heavily optimized **WinTUN** ring-buffer adapter. Route management is enacted directly via the Win32 API, and DNS leaks are violently prevented using the Windows **Name Resolution Policy Table (NRPT)**, forcing the OS to forward all DNS queries through the tunnel regardless of interface metric.
- **Linux (`linux` crate)**: Directly interacts with `/dev/net/tun` via low-level `ioctl` calls using the `tun` abstraction. It employs `systemd-resolved` interfacing to push the VPN DNS resolver to the top of the OS lookup table.
- **Android (`android` crate)**: Eliminates the Java Native Interface (JNI) overhead by directly passing the file descriptor of the Android `VpnService` into the Rust asynchronous event loop via `tokio::io::unix::AsyncFd`. Packet polling is entirely offloaded to native Rust epoll queues, keeping the JVM completely out of the datapath.

3 Transport & Framing: The QUIC Paradigm

Operating a Layer 3 VPN over TCP leads to catastrophic "TCP Meltdown" when packets are lost. Operating over raw UDP leaves the implementation solely responsible for congestion control and encryption.

Mavi VPN leverages **QUIC**. We use the `quinn` Rust crate (implementation of IETF QUIC) to provide the best of both worlds: independent, unreliable datagrams encapsulated in a state-of-the-art encrypted, congestion-controlled wrapper.

3.1 Datagrams vs. Streams

- **The Data Plane (Datagrams)**: Encapsulated IP packets (IPv4/IPv6) are transmitted using QUIC Datagrams (RFC 9221). Because datagrams are unordered and unreliable, the loss of one packet does not delay subsequent packets, completely solving the HoL blocking problem while allowing the inner OS TCP stack to handle retransmissions correctly.

- **The Control Plane (Streams)**: Configuration, Handshake (Auth messages), and telemetry are sent over QUIC Streams. This guarantees that initial network assignments (Gateway IPs, Routes, MTU) are delivered with cryptographic certainty before the client configures its virtual interface.

3.2 Congestion Control: BBR (Bottleneck Bandwidth and RTT)

Traditional CUBIC congestion algorithms interpret packet loss as network congestion and instantly halve transmission velocity. On erratic mobile or international links, this leads to chronic under-utilization. Mavi VPN explicitly initializes the QUIC transport configuration to utilize Google's **BBR** congestion control. BBR continually models the actual bandwidth pipe and baseline RTT, ensuring high-speed throughput even over wireless topologies exhibiting 5%-10% innate packet loss.

4 Censorship Resistance & DPI Evasion

Sophisticated firewalls currently deploy heuristic-based Deep Packet Inspection, utilizing entropy analysis and handshake profiling to sever unauthorized protocols. Mavi VPN integrates multiple countermeasures to neutralize these threats.

4.1 Encrypted Client Hello (ECH - RFC 9180)

In standard TLS 1.3, the Server Name Indication (SNI) is transmitted in plaintext, allowing firewalls to block connections to known VPN endpoints instantly.

- Mavi VPN implements **ECH GREASE** capability on the client.
- The client parses an administrator-provided `ECHConfigList` (a base64/hex payload containing the server's HPKE X25519 public key and symmetric ciphers).
- Utilizing the 'rustls' framework with the 'aws-lc-rs' backend, the client crafts an *Outer* ClientHello containing a benign cover domain (e.g., `cloudflare-ech.com`), whilst encrypting the actual *Inner* ClientHello.
- Censors observing the packet stream perceive only a standard connection request to a major CDN infrastructure.

4.2 Active Probe Emulation (The Fake Nginx Server)

If a DPI system (or a malicious actor) strips the ECH payload, guesses the endpoint, or initiates a raw scan on the UDP port, a standard VPN would drop the packet or issue a QUIC `CONNECTION_CLOSE`, which immediately identifies it as a non-standard service.

Mavi VPN performs **Active Probe Emulation**. When an incoming connection fails authentication or presents an invalid ALPN, the backend does not silently drop it. Instead:

1. A grace period of 50ms (`H3_DETECTION_GRACE`) is allocated to allow stream multiplexing.
2. The engine shifts the connection context natively into an `h3` (HTTP/3) handler.
3. It creates a control stream (`0x00 0x04 0x00`) representing HTTP/3 settings.
4. It sends raw, QPACK-encoded headers equivalent to `:status 200` and `server: nginx`.
5. It flushes an HTML layout representing the default "Welcome to nginx!" landing page directly onto the bidirectional QUIC stream.

This sophisticated behavior tricks even the most advanced DPI scanners into cataloging the server as a benign, standard web server.

4.3 MASQUE (RFC 9484) Connect-IP Encapsulation

When extreme stealth is required, the "HTTP/3 Framing" flag forces the VPN into strict compliance with the MASQUE network framework.

1. The client establishes a genuine HTTP/3 Request over a bidirectional QUIC stream: `CONNECT /.well-known/masque/ip/*/*/ HTTP/3` with `:protocol=connect-ip`.
2. The tunnel configuration is not sent via custom bytes, but encapsulated within proper HTTP/3 **Capsules**. Mavi utilizes `ADDRESS_ASSIGN` (0x01) and `ROUTE_ADVERTISEMENT` (0x03) exactly as specified by IETF to allocate IPs.
3. A proprietary vendor capsule, `MAVI_CONFIG` (0x4D56), transmits supplementary configuration (DNS parameters, Pinned MTU metadata).
4. Data datagrams are explicitly prefixed with the RFC 9484 Quarter Stream ID and Context ID (`0x00 0x00`), ensuring the raw binary representation exactly mirrors IETF proxy standards.

5 Authentication and Cryptography

Mavi VPN entirely eschews Public Key Infrastructure (PKI) and Certificate Authorities (CAs). Relying on Root CAs leaves clients vulnerable to state-sponsored interception if a state CA is compromised or force-installed on a corporate machine.

5.1 SHA-256 Certificate Pinning

- During backend deployment, the server generates a unique TLS 1.3 certificate alongside its private X25519 key.
- It immediately computes the SHA-256 fingerprint hash of the DER-encoded certificate and stores it in `cert_pin.txt`.
- The client config incorporates this hex pin. Upon connecting, the client's internal `rustls::client::Serve` directly intercepts the certificate chain. If the computed hash of the leaf certificate does not perfectly match the pin, the connection is instantly aborted with an `UnknownIssuer` fatal error. This guarantees absolute immunity against MitM interception.

5.2 Identity Provider Integration (Keycloak OIDC)

While static tokens are suitable for private deployments, Mavi VPN's architectural scale targets enterprise environments requiring complex Identity and Access Management (IAM).

- The platform integrates native **Proof Key for Code Exchange (PKCE) OAuth2** flows.
- The Tauri GUI or CLI initializes an ephemeral micro-webserver on `localhost:18923` and spawns the machine's default browser targeting the Keycloak authorization endpoint.
- Upon successful SSO login, the IdP redirects back to the micro-server, passing JWT Authorization codes.
- The VPN backend securely validates the resulting JSON Web Token against Keycloak's cryptographic JWKS (JSON Web Key Set) endpoint, enforcing `azp` (Authorized Party) checks in constant time.

6 The "Pinned MTU" Engineering Strategy

The Maximum Transmission Unit (MTU) represents the single most significant friction point in VPN engineering. Exceeding path MTU leads to dropped frames and "Black Holing".

6.1 Evaluating Path MTU Discovery (PMTUD)

QUIC traditionally attempts to increase probe sizes to discover the maximum available datagram size. On modern residential lines (DSL, Fiber via PPPoE, DS-Lite tunnels), the MTU frequently dips to 1460-1492 bytes. If 1500-byte encapsulated packets are transmitted, upstream routers must fragment them. Due to security heuristics, most ISPs aggressively drop fragmented UDP traffic, triggering silent protocol collapse.

6.2 Dual-Boundary Pinned Limitation

Mavi VPN abolishes PMTUD variance in favor of strict "Pinned Mode".

1. The Outer Wire Tunnel: Pinned at 1360 Bytes

We actively disable PMTUD within the Quinn stack:

```
transport_config.mtu_discovery_config(None);
transport_config.initial_mtu(1360);
transport_config.min_mtu(1360);
```

At 1360 bytes, the packets are substantially smaller than even the most severely constrained PPPoE gateways (1460 bytes). Because the packet never challenges the infrastructure's MTU ceiling, fragmentation is structurally impossible.

2. The Inner Payload Tunnel: Pinned at 1280 Bytes

1280 Bytes is the absolute minimum MTU acceptable for an IPv6-compliant network stack. By configuring the local virtual TUN adapter to 1280 MTU, the Windows/Linux/Android kernel will *never* schedule an IP packet larger than 1280 bytes into the VPN driver.

3. The Protocol Mathematics

- Allowed Transmission Array: 1360 Bytes.
- Transport Overhead: ∼80 Bytes (40B for IPv6 + 8B for UDP + 32B for QUIC Headers/TLS Auth Tag).
- Available Payload: $1360 - 80 = 1280$ Bytes.
- Max Inner Packet: 1280 Bytes.

The payload perfectly occupies the safe transport array, ensuring zero-fragmentation tunneling on a global scale.

6.3 ICMP Asynchronous Emulation

In edge cases where a local application illegally ignores the MTU of the virtual TUN device, the Mavi VPN core intercepts the oversized packet rather than blindly forwarding it. The core extracts the source/destination IPs and dynamically synthesizes an **ICMPv4/ICMPv6 "Packet Too Big" (PTB)** error payload directly inside the event loop, immediately injecting it back into the local operating system's ingress queue. This forces the host OS to accurately clamp its TCP Maximum Segment Size (MSS) without requiring messy server-side `iptables mangle` hacks.

7 System Optimization and Kernel Offloading

To saturate gigabit uplinks, Mavi VPN engages a multitude of hyper-optimizations.

7.1 Buffer Inflation and GSO

Standard UDP sockets exhibit shallow queueing buffers. This is detrimental when handling multi-megabyte cryptographic bursts.

- Mavi VPN enforces 4 MB `SO_SNDBUF` and `SO_RCVBUF` arrays.
- **Generic Segmentation Offload (GSO)** is explicitly instructed on the transport configuration. This permits the Rust application to present large payload segments to the Linux kernel as a single write, offloading the physical segmentation geometry to the Network Interface Card (NIC) silicon, vastly reducing CPU cycle expenditures per instruction.

7.2 IpGuard RAII and Dual-Stack Operations

Server-side memory and IP tracking are strictly safeguarded by RAII (Resource Acquisition Is Initialization) semantics. When an identity token is authenticated, an `IpGuard` struct secures an IPv4 and IPv6 subnet from the central `DashMap`. When the QUIC session terminates—be it through a clean disconnect or a catastrophic timeout—the `IpGuard` is automatically dropped out of scope, securely releasing mathematical leases back to the pool instantly. The network natively supports **Dual-Stack** deployments, assigning a dedicated `fd00::/64` IP and routing it accurately using `ip6tables` NAT66 masquerading to the WAN block.

8 Conclusion

The Mavi VPN architecture represents a decisive paradigm shift in secure network tunneling. By transitioning away from easily identified raw network protocols and encapsulating traffic securely within authentic HTTP/3 streams, it nullifies modern Deep Packet Inspection perimeters. Through meticulous memory handling, cryptographic enforcement (ECH, Certificate Pinning), and statistically impervious MTU pinning, Mavi VPN delivers robust, high-performance edge routing designed explicitly to withstand the aggressive censorship behaviors of the contemporary internet landscape.